Modular Programming :-

Let 'T' be the name of a data type.

Modular programming encourages splitting the implementation of T into three files.

1) T.h : Header file inclusions, #defines, typedefs, struct T definition, declarations of all functions managing struct T instances.

2) T.c : Definition of all functions managing struct T instances.

3) useT.c : Using struct T instances via management functions.

Both T.c and useT.c will include T.h

Advantage 1 : As implementation of type T is isolated from its use, we can compile T.h and T.c into a static or dynamic link library.

T.h + T.lib (or T.h + T.c) = MODULE [REUSABILITY]

Advantage 2 : If we upgrade the definition struct T, and the definitions of functions manging struct T instances, keeping their declarations intact, then we can upgrade type T without disturbing the client.

[EXTENDABILITY].

```
struct Date
{
    int day;
    int month;
    int year;
};
```

TERMINOLOGY:

In C, we use struct to implement what are known as aggregate types. The types for which a single instance of built in data type such as char, int, float is not sufficient.

```
    float income_tax;
    float interest_rate;
    int height_ft;
```

Aggregate type is made of any number of members with different types.

```
    struct T
    {
        T1 mem1;
        T2 mem2;

        Tn mem-n;
    };

    struct Date
    {
        int day;
        int month;
        int year;
    };
```

One instance of struct Date is made up of three members

```
Member 1-> day, type -> int
Member 2-> month, type -> int
Member 3-> year, type -> int
```

} layout information

[Refer to next page]

Structure of Date.h file

```
struct Date
{
    // LAYOUT OF DATE
};

// Declarations of functions which manage instances of struct Date
// Management = CreateInstance + ReleaseInstace + UseInstance
```

C++ is backward compatible language with C.
So why C++ invented a new statement viz. class statement to define a new data type? (when struct was available)

Reason 1: With struct statement, there is NO SYNTACTIC WAY to DEPICT RELATIONSHIP BETWEEN THE DAYA LAYOUT AND FUNCTIONS MANAGING DATA LAYOUT

```
struct Date
{
    int day;
    int month;
    int year;
};
```

→ layout information

```
struct Date* getDateInstance(int, int, int);
int get_day(struct Date*);
.
.
.
void releaseDateInstance(struct Date*);
```

→ functions managing instances created from the layout.

struct statement in C does not allow function definition inside it.
And therefore, there is no way to depict relationship between data layout of a data type and functions managing instances created from that layout.

Reason 2: In C programming languages, whichever function creates an instance of a structure will have access to all members of structure.

```
struct Date
{
    int day, month, year;
};
```

```
int main(void)
{
    struct Date myDate_1;

    // main function can access day, month, year members of instance myDate_1
    myDate_1.day;
    myDate_1.month;
    myDate_1.year;
}

void test(void)
{
    struct Date* pDate = (struct Date*)malloc(sizeof(struct Date));

    pDate->day;
    pDate->month;
    pDate->year;
}

Root cause analysis:
    If you want modular programming to yield advantages of extendability
    and reusuability then only management function should be able to access
    the internal members of instance created from data type. Rest of the
    functions should be able to create the instance but should not be able
    to access the internal data members so that they will be forced to
    "use" the instance via management functions.
    But the rule of C that 'whichever function that has access to
    any instance of the struct will be able to access all its internal members'
    does not help!

#--------------------

C with classes.

#--------------------

What can be written inside class.
    1)  Data definition statement (data member)
    class X1
    {
        int num;  // data definition statement
    };
```

```
2) Static data definition statement (static data member)
class X2
{
    static int num;
};
int X2::num;

3) Function (member function)
class X3
{
    void get_num()
    {
    }
};

4) Static function (static memmber function)
class X4
{
    static int get_next_employee_id()
    {

    }
};

5) typedefs
class X5
{
    typedef int day_t;
    typedef int pid_t; // etc
};

6) Nested class
class X
{
    class Y
    {
    };
};
```

Inside class, we can define
(1) data member (2) static data member (3) member function (4) static member function (5) typedefs (6) nested class .

C – struct

```
struct Date
{
    int day;
    int month;
    int year;
};

struct Date myDate;
```

year	4
month	4
day	4

myDate

```
int main(void)
{
    myDate.day = 28;      ✓
    myDate.month = 9;     ✓
    myDate.year = 2025;   ✓
}
```

C++ class.

```
class Date
{
private:                  → Access Specifier
    int day;
    int month;
    int year;
};

Date myDate;
```

year	4
month	4
day	4

myDate

```
int main(void)
{
    myDate.day = 28;      ✗
    myDate.month = 9;     ✗
    myDate.year = 2025;   ✗
    Date* pDate = &myDate;
    pDate->day;           ✗
}
```

```
*(int*)((char*)&myDate + 0)
*(int*)((char*)&myDate + 4)
*(int*)((char*)&myDate + 8)
```

ADV. USE.